

AMSS 2026 — Lecture 5: Other Structural Views

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: from one class diagram to the whole structure
2. Live opener: drive AI to a package/component view
3. Reading floor: object, package, component (deployment in passing)
4. How AI over- and under-decomposes
5. Reading critically: the architecture read-order
6. Bridge to Lab 3 + Week 6

Recap: Architect and Critic

- ▶ **Architect / director:** drive AI through the software development lifecycle (SDLC).
- ▶ **Critic / reviewer:** read AI's output and name what's wrong or missing.

In Week 4 you critiqued an AI-drawn class diagram. Today: the same loop, one level up — at the architecture.

From One Class Diagram to the Whole Structure

- ▶ **Week 4 gave you one class diagram** — User, Rental, Bike, Station, Payment and how they relate.
- ▶ **A real system has many such classes**, across many parts.
How is it carved up?

Today's question: **what are the parts, and is the split right?**

The Literacy Floor: Critique, Rationale, Traceability

From Week 1: in the oral defense, *unaided*, you must demonstrate:

- ▶ **Critique** — read & critique AI-generated artifacts on the spot.
- ▶ **Rationale** — articulate why you directed AI a certain way.
- ▶ **Traceability** — defend traceability across your project.

Today is Critique at the architecture level: read an AI-drawn structure and name where it over- or under-decomposes.

Four Views, One Question

All four answer “what are the parts?” at different altitudes:

- ▶ **Object** — a *snapshot*: specific instances at one instant.
- ▶ **Package** — a *logical grouping*: which classes belong together.
- ▶ **Component** — a *replaceable unit* with declared interfaces.
- ▶ **Deployment** — *where it runs*: nodes and artifacts.

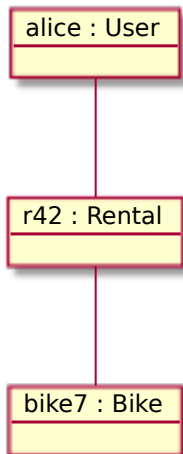
Today's load-bearing pair: **package + component**. The other two: recognise them.

Demo: Drive AI to an Architecture View

Live: ask AI to decompose the bike-sharing system into packages and components. Watch the grain — too many parts, or invented ones.

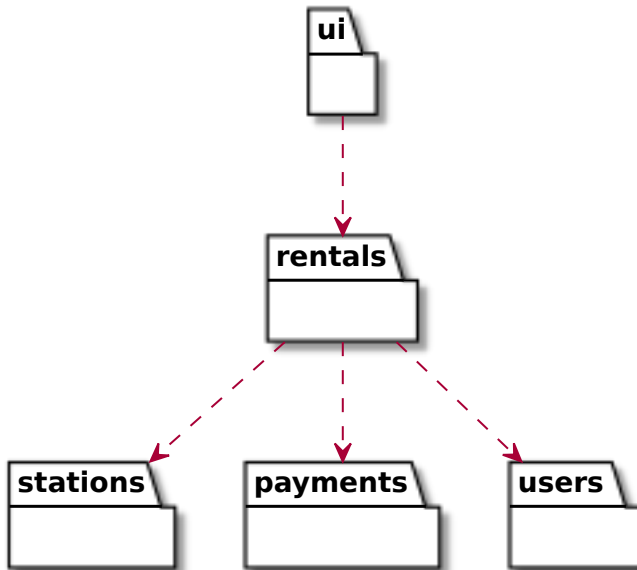
Prompt to AI: “Generate a UML component diagram (as PlantUML) for the city bike-sharing app from last week: rentals, stations, payments, and users.”

Object Diagram: A Snapshot



The class diagram's *types* at one **instant**: alice holds rental r42 on bike7.

Package Diagram: Grouping



A package groups related classes. The dashed arrow reads

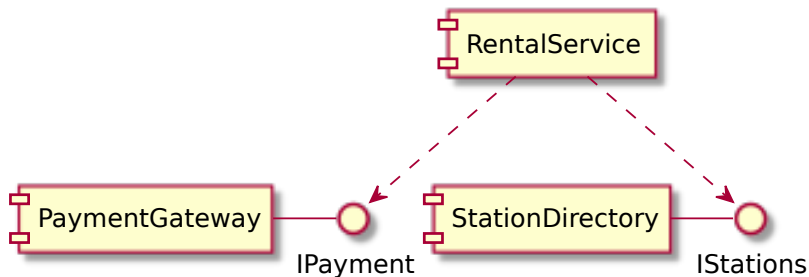
Reading a Package Diagram

Read every dependency aloud, and ask two things:

1. **Direction** — do dependencies flow one way (ui toward domain), or loop back?
2. **Cycles** — does any package depend, directly or transitively, on one that depends on it?

A dependency **cycle** is a smell: the two packages are really one.

Component Diagram: Replaceable Units



PaymentGateway **provides** IPayment (lollipop); RentalService **requires** it (dashed). A component is defined by its **interfaces**, not its internals.

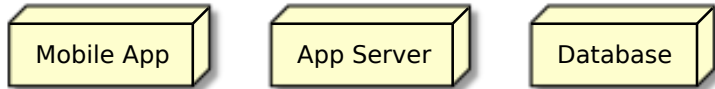
Reading a Component Diagram

For each component, ask:

1. What does it **provide**? (the interfaces others depend on)
2. What does it **require**? (the interfaces it depends on)
3. Could you **replace** it without touching its neighbours?

If you can't name a component's provided and required interfaces, it isn't a component.

Beyond the Floor: Deployment



Mobile App" --> "App Server



App Server" --> "Database

Where it runs: nodes (hardware/environments) and the artifacts on them. It matters for non-functional requirements — latency, availability — but it is not today's drill.

AI Decomposes Plausibly — and Gets the Grain Wrong

A package or component diagram can look professional and still split the system badly. The critic question:

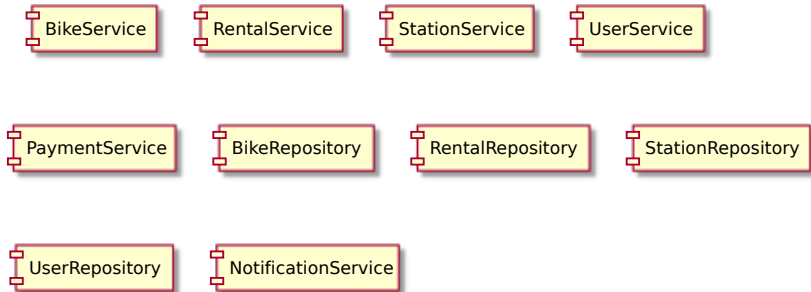
“Are these the right parts, at the right grain — or just boxes?”

Defect #1: Under-Decomposition



One package — or one BikeShareApp component — holding everything. **Critique:** *“Where are the boundaries? Name three parts that could be built and tested separately.”*

Defect #2: Over-Decomposition



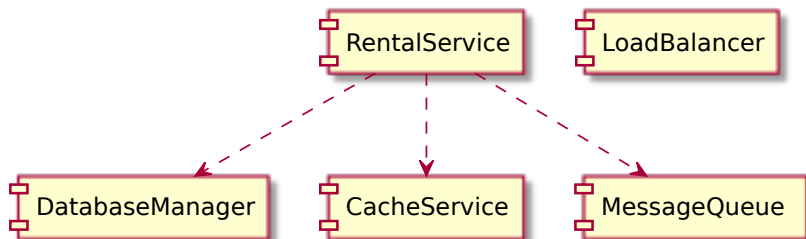
A component **per class**, plus a repository per entity.

Your Turn: One Part or Three?

With your neighbour (60s): pick which **two of these boxes are really one part** — vote, then we critique.

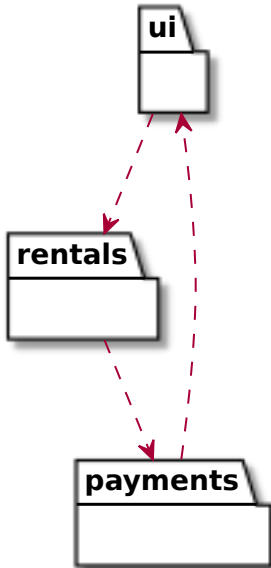
Critique (the reveal): *“What boundary does each split earn? Which of these are really one part?”*

Defect #3: Invented Infrastructure



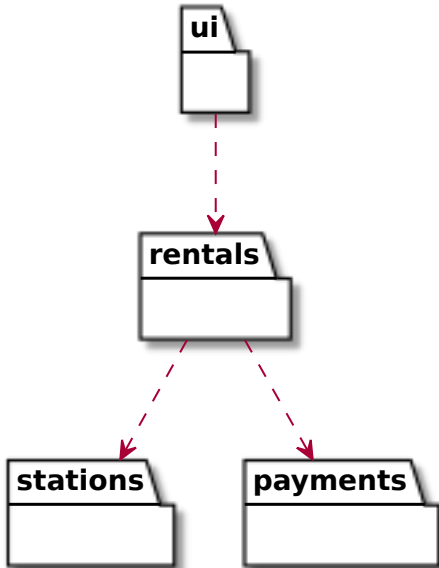
`DatabaseManager`, `CacheService`, `MessageQueue` — infrastructure nobody asked for. **Critique:** *“Is this in the requirements, or did AI assume an architecture?”*

Defect #4: Wrong Dependency Direction



payments depends back on **ui** — a **cycle**, and inverted layering.

The Critic Re-Decomposes



A handful of **domain** packages; dependencies flow **one way**; no

How to Read an AI Architecture View

A fixed order, every time:

1. Are these the real **parts** of *this* system? (domain, not invented infrastructure)
2. Is the **grain** right? (not one box; not a box per class)
3. Do **dependencies** point one way? (no cycles, layering respected)
4. Is each **component** defined by its interfaces? (provided + required)

The Critique Loop on Architecture

read -> name the bad split -> re-prompt with structural constraints
-> re-read.

Same architect-and-critic loop as Week 2 (requirements), Week 3 (tests), Week 4 (class diagram) — now on a decomposition.

The Human Decides the Right Grain

AI can split a system into one box or a hundred. Neither is “more correct” on its own — the **right grain** is a judgment about *this* system’s real boundaries.

That judgment is the architect-and-critic call, and AI can’t make it for you.

Looking Ahead: Lab 3 (Next Week)

Lab 3 is a **team defect hunt**: you'll receive flawed structural artifacts — a class diagram, a package diagram, a component diagram — and compete to find the most defects, with severity ratings.

Today's read-order is your defect-hunt rubric. This lecture was the dry run.

Next Week: Behavioral I (Week 6)

Structure is done — Week 4 (classes) and Week 5 (architecture) cover *what the parts are*.

Week 6: use cases and sequence diagrams — *how the parts interact over time*, and where AI fabricates messages.

Critique, Rationale, Traceability — The Through-Line

Today you drilled:

- ▶ **Critique** — read an AI architecture view and named where it over- or under-decomposed.
- ▶ **Traceability** — the package/component is a structural node in the trace: requirement -> use case -> class -> part.

Rationale (why you directed AI a certain way) lands in your project narrative.

That's It For Today

- ▶ Next lecture (Week 6): behavioral I — use cases + sequence.
- ▶ Next week's lab (Lab 3): team defect hunt on flawed structural artifacts.

Questions?